SwapUni

A Campus Marketplace Desktop Application

Java Swing | Spring Boot | MySQL

Project Report

BCA 3C - Java Project

Group No.	8
Team members (Roll No.)	Shruti Mishra (92500527147) Kinjalba Jadeja (92500527125)
Guided by	Tirth Bhadeshiya
Institute	MARWADI UNIVERSITY,Rajkot
Academic year	2026-27

Contents

1. Abstract
2. Introduction and Objectives
3. Technology Stack
4. System Architecture
5. Modules and Features
6. Database Design
7. REST API Summary
8. Product Photo Upload (design)
9. User Interface and Screenshots
10. Testing
11. Installation and Running
12. Limitations and Future Scope
13. Conclusion

1. Abstract

SwapUni is a campus marketplace for college students. Students often own books, calculators, lab equipment and stationery that they no longer need, while other students need exactly those items. SwapUni lets students list such items and buy, sell, exchange, borrow or donate them within their own campus. It is built as a Java Swing desktop application that talks to a Spring Boot REST backend over HTTP; the data is stored in a local MySQL database. Both the backend and the desktop window start from one program, so the user only runs a single application.

2. Introduction and Objectives

Second-hand trading between students usually happens through scattered chat groups, where listings are hard to search and easy to miss. A dedicated, secure and searchable marketplace solves this problem.

Objectives

- Provide secure sign-up and login with hashed passwords.
- Let students list items with details and a photo, and edit or delete their own listings.
- Support four transaction types: Sell, Exchange, Borrow and Donate.
- Allow requests between students with accept / reject / cancel and an automatic transaction history.
- Offer search, category filters, favourites and a personal dashboard.
- Demonstrate a layered architecture (GUI, controller, service, repository, database) using Java 17 and Spring Boot.

3. Technology Stack

Layer	Technology	Purpose
Language	Java 17	Whole application
Desktop GUI	Java Swing (custom rounded components)	Screens, forms, dialogs
Backend	Spring Boot 3.2.5, Spring Web	REST API and business logic
Persistence	Spring Data JPA / Hibernate	Object-relational mapping, repositories
Database	MySQL 8	Permanent data storage
Security	BCrypt (spring-security-crypto)	Password hashing; token based sessions
Client HTTP / JSON	java.net.http.HttpClient, Jackson	GUI to backend communication
Images	javax.imageio, webp-imageio	Reading and previewing JPG, PNG, GIF, WEBP
Build tool	Gradle	Build, run, packaging

4. System Architecture

The application follows a layered design. The Swing GUI never touches the database; it only calls the REST API. Each layer has one responsibility, which makes the code easy to read, test and extend.

Layer	Package	Responsibility
Presentation	com.uniswap.gui	Swing screens, dialogs, reusable UI components, API client
Controller	com.uniswap.controller	REST endpoints, request validation
Service	com.uniswap.service	Business rules (ownership checks, status changes, file storage)
Repository	com.uniswap.repository	Database queries with Spring Data JPA
Model / DTO	com.uniswap.model, dto	Entities (tables) and request / response objects
Config / Exception	com.uniswap.config, exception	Sample data, static file serving, clean error messages

Flow of one action (example: saving a listing): Add Listing dialog -> ApiClient (HTTP) -> ListingController -> ListingService -> ListingRepository -> MySQL, and the result travels back the same way as JSON.

5. Modules and Features

Module 1 - Authentication and Profile

Sign up, log in, log out, view and edit profile. Email and username are unique; passwords are stored only as BCrypt hashes.

Module 2 - Marketplace / Listings

Browse all active items, search by keyword, filter by category and transaction type, open full details, and add, edit or delete own listings. Each listing can have its own photo.

Module 3 - Requests

A student can send a buy, exchange or borrow request on any item. The owner sees incoming requests and can accept or reject; the requester sees outgoing requests and can cancel a pending one. Accepting a request records a transaction and updates the listing status.

Module 4 - Favourites

Save items to a wishlist, remove them later and open their details. Duplicate favourites are prevented.

Module 5 - My Listings and Transactions

A screen with all items the user listed (active, sold, exchanged, borrowed, donated) and a history of completed transactions.

Module 6 - Dashboard

Welcome message, counts of active listings, favourites and pending requests, quick-action buttons, popular categories and recent marketplace items.

6. Database Design

The database uniswap_db has six tables. Hibernate creates and updates them automatically (ddl-auto=update), and database/uniswap_db.sql is included as a reference script.

Table	Important columns	Notes
users	id, full_name, email, username, password_hash, course, study_year, campus, bio, created_at	email and username are unique
categories	id, name	Books, Electronics, Stationery, College Supplies, Lab Equipment, Other
listings	id, name, description, category_id, price, item_condition, transaction_type, status, owner_id, location, image_url, created_at	image_url stores the photo link (NULL = no photo)
favourites	id, user_id, listing_id, created_at	unique (user_id, listing_id)
requests	id, listing_id, requester_id, type, message, extra_detail, status, created_at, updated_at	status: PENDING, ACCEPTED, REJECTED, CANCELLED, COMPLETED
transactions	id, listing_id, requester_id, owner_id, type, amount, status, completed_at	created when a request is accepted

Relationships: one user owns many listings; one category has many listings; a favourite links one user to one listing; a request links a requester to a listing; a transaction links the requester, the owner and the listing.

Enumerations: Transaction type = SELL, EXCHANGE, BORROW, DONATE. Listing status = ACTIVE, SOLD, EXCHANGED, BORROWED, DONATED, INACTIVE. Item condition = NEW, LIKE_NEW, GOOD, FAIR.

7. REST API Summary

Method	Endpoint	Purpose
POST	/api/auth/signup, /api/auth/login, /api/auth/logout	Account creation and session
GET / PUT	/api/auth/profile	View and update profile
GET	/api/dashboard	Dashboard numbers and recent items
GET	/api/categories	Category list
GET	/api/listings?keyword=&categoryId=&type=	Search and filter active listings
GET	/api/listings/mine, /api/listings/{id}	Own listings, one listing
POST	/api/listings/upload-image	Upload a product photo (multipart), returns its URL
POST / PUT / DELETE	/api/listings, /api/listings/{id}	Create, edit, delete a listing
PUT	/api/listings/{id}/status	Mark sold / exchanged / borrowed / donated
GET / POST / DELETE	/api/favourites, /api/favourites/{listingId}	Wishlist
POST	/api/requests	Send a request
GET	/api/requests/incoming, /api/requests/outgoing	Received and sent requests
PUT	/api/requests/{id}/accept, /reject, /cancel	Change request status
GET	/api/transactions	Transaction history

8. Product Photo Upload (design)

Every listing can have its own photo. The upload works in two steps:

- The Add / Edit Listing dialog has a file picker. The chosen image is checked on the client (type JPG, JPEG, PNG, WEBP or GIF; size up to 5 MB; readable) and shown as an immediate preview.
- When the form is saved, the file is sent to POST /api/listings/upload-image as multipart/form-data. The server checks the real file bytes (not just the name), stores it under a random name in the upload folder and returns its URL.
- The URL is saved in listings.image_url together with the listing. On Edit, a new upload replaces the old photo; if no new file is chosen the current photo is kept.
- Photos are served from /uploads/** and displayed on the Marketplace, Dashboard, My Listings, Favourites and Item Details screens. Items without a photo show a coloured placeholder.

Because every file gets a unique random name, replacing one product's photo can never overwrite another product's photo. A failed or invalid upload shows a clear message and the listing is not saved.

9. User Interface and Screenshots

The interface uses the SwapYuni design: a dark slate sidebar, burgundy primary buttons, terracotta secondary buttons, sage and forest accents and warm parchment / eggshell backgrounds. Product cards and listing rows reflow into columns according to the window width, and every window can be resized or maximized. Screenshots were taken with sample data.

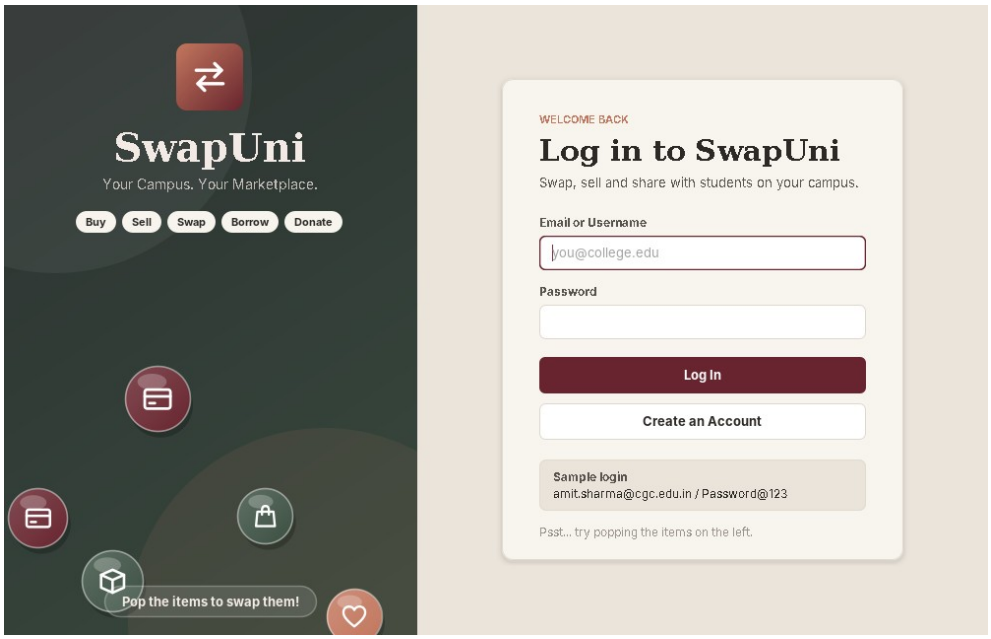


Figure 1: Login screen with the "Pop the Swap" click game

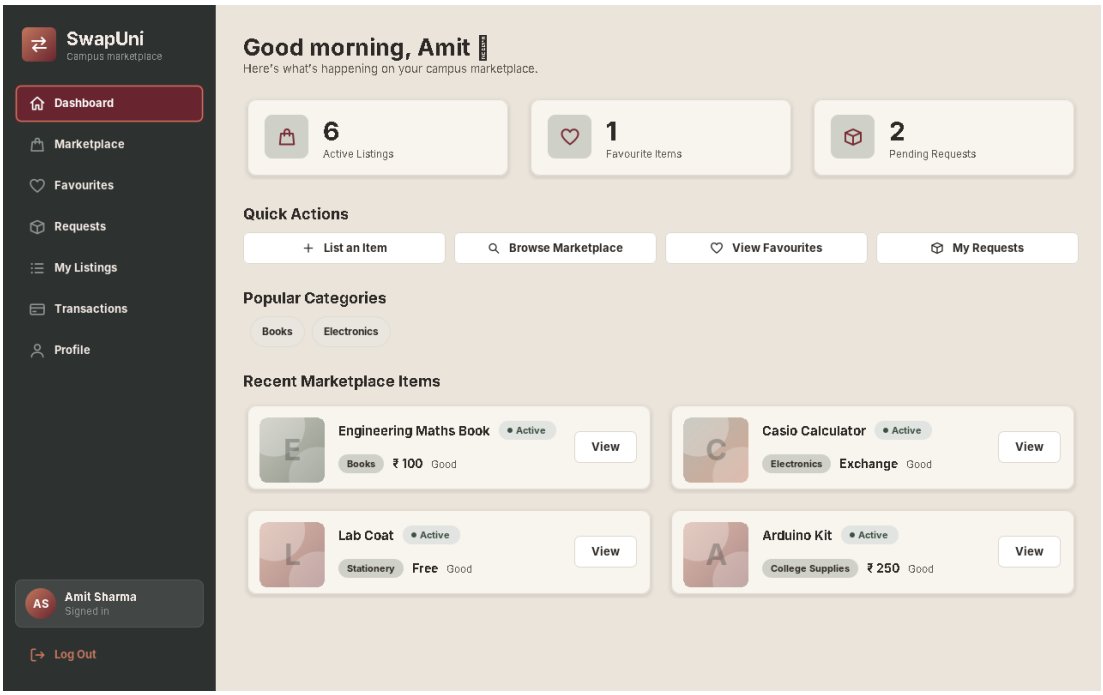


Figure 2: Dashboard

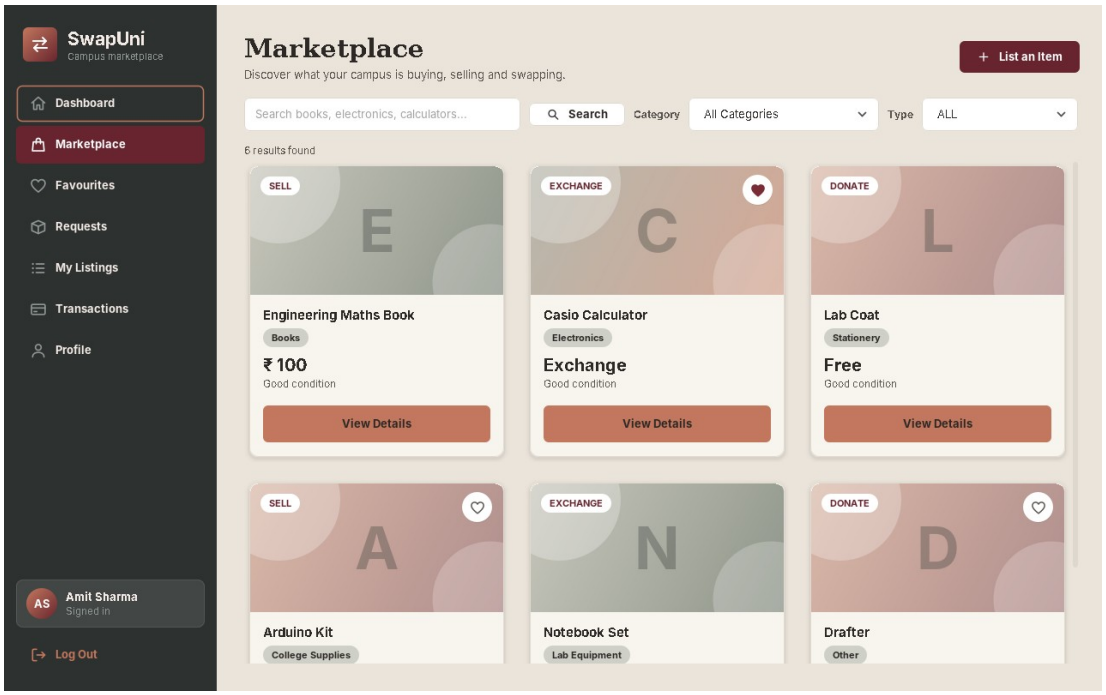


Figure 3: Marketplace with product cards in a responsive grid

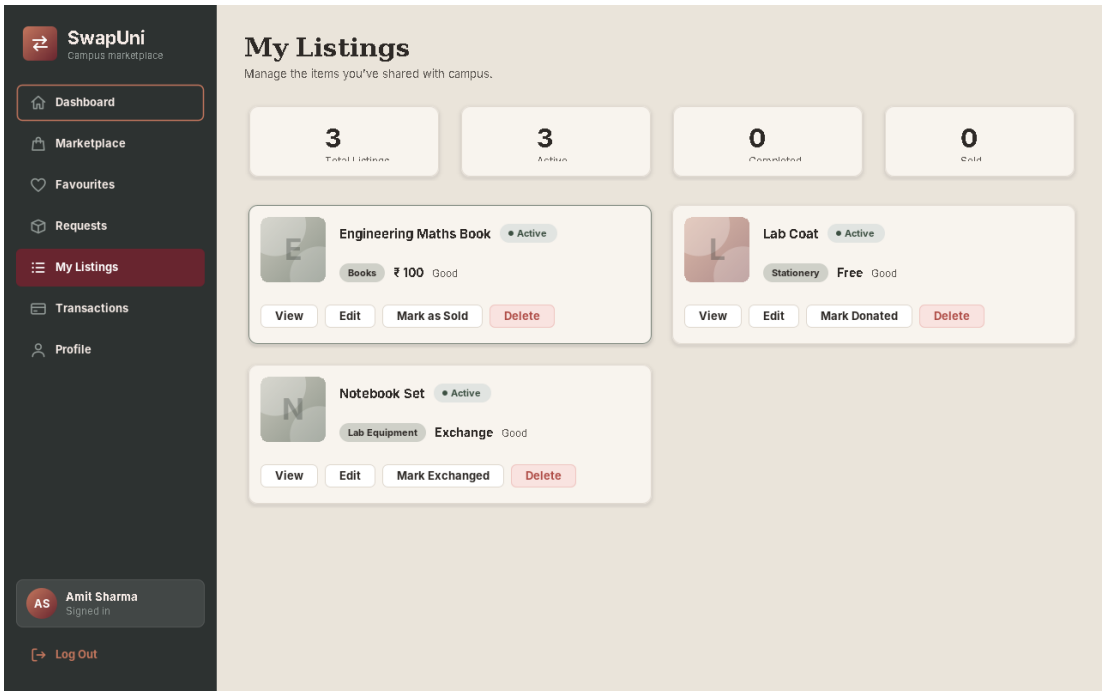


Figure 4: My Listings

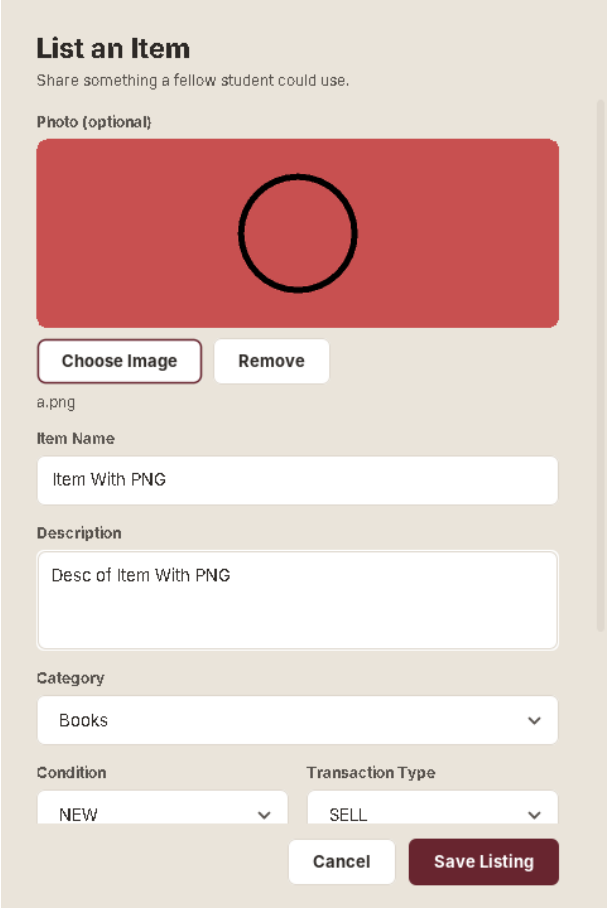


Figure 5: Add Listing with photo picker and live preview

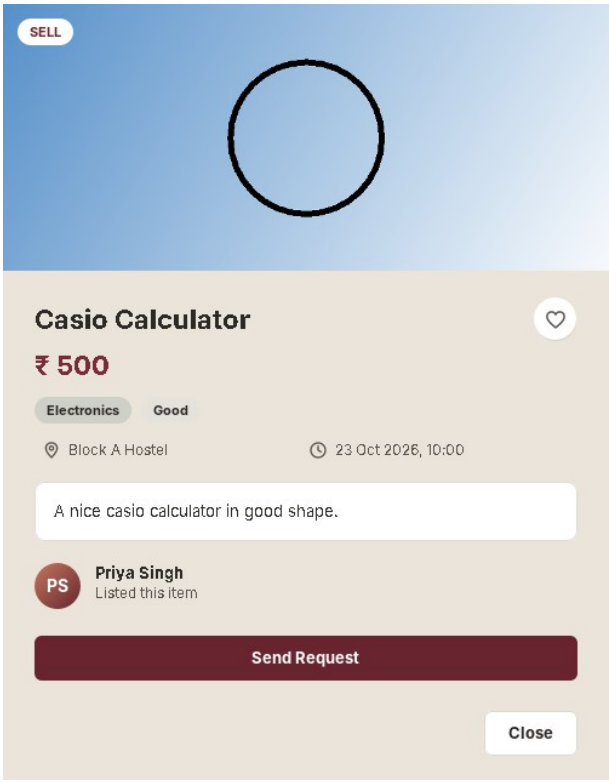


Figure 6: Item Details with photo

10. Testing

The following checks were carried out:

Area	Check	Result
Photo upload	Valid JPG, PNG, GIF accepted; multipart upload; URL saved with the right listing	Passed
Photo upload	Wrong file type, corrupted image and a file over 5 MB are rejected with a message	Passed
Photo upload	Editing keeps or replaces the photo; another product's photo is never changed	Passed
Photo upload	Failed upload shows an error and the listing is not created	Passed
Server storage	Extension and real-content checks, 5 MB limit, unique names, no path traversal	Passed (unit test)
Layout	Cards reflow into columns when the window is resized; Login / Sign Up windows can be maximized	Passed
Functional	Login, sign up, add / edit / delete listing, favourites, requests, dashboard	Checked by running the application

11. Installation and Running

1. Install JDK 17 or newer, Gradle and MySQL 8, and make sure MySQL is running.
2. Open PowerShell in the code folder (the one containing build.gradle).
3. Set the database login: $env:DB_USERNAME="root" and $env:DB_PASSWORD="<your password>".
4. Run: gradle bootRun. The database and tables are created automatically.
5. Log in with the sample account amit.sharma@cgc.edu.in / Password@123, or create a new account.
6. Optional: double-click Create-SwapUni-Shortcut.bat to build SwapUni.exe and a Desktop shortcut.

12. Limitations and Future Scope

- The desktop client talks to a backend on localhost, so each computer runs its own copy; a shared server would let many students use one marketplace.
- In-app chat between buyer and seller and notifications for new requests.
- Multiple photos per listing and image compression.
- Email verification and a password-reset flow.
- Online payment integration and ratings / reviews for users.

13. Conclusion

SwapUni shows how a complete, layered Java application can be built with Swing, Spring Boot and MySQL. It covers authentication, CRUD operations, searching and filtering, file upload, request workflows and a responsive desktop interface, and it solves a real problem for campus students.